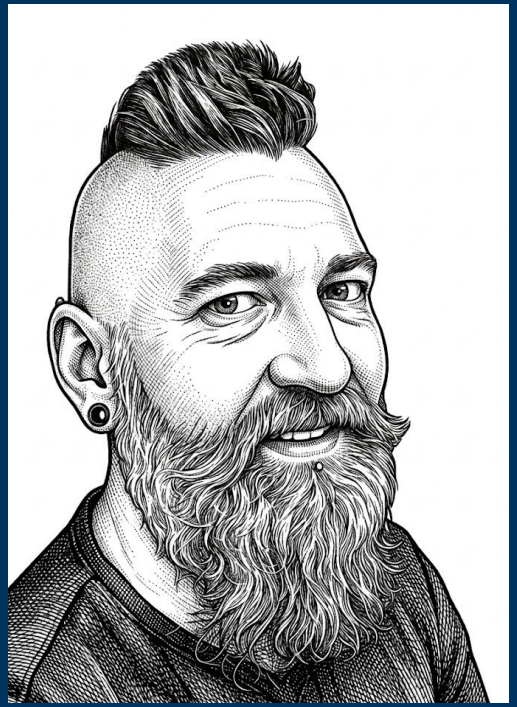

Kill your Tests
Wie wir den E2E-Sumpf mit KI-Agenten endlich trockenlegen
NÜRNBERGER
VERSICHERUNG
© felix tensing

NÜRNBERGER VERSICHERUNG



Felix Tensing

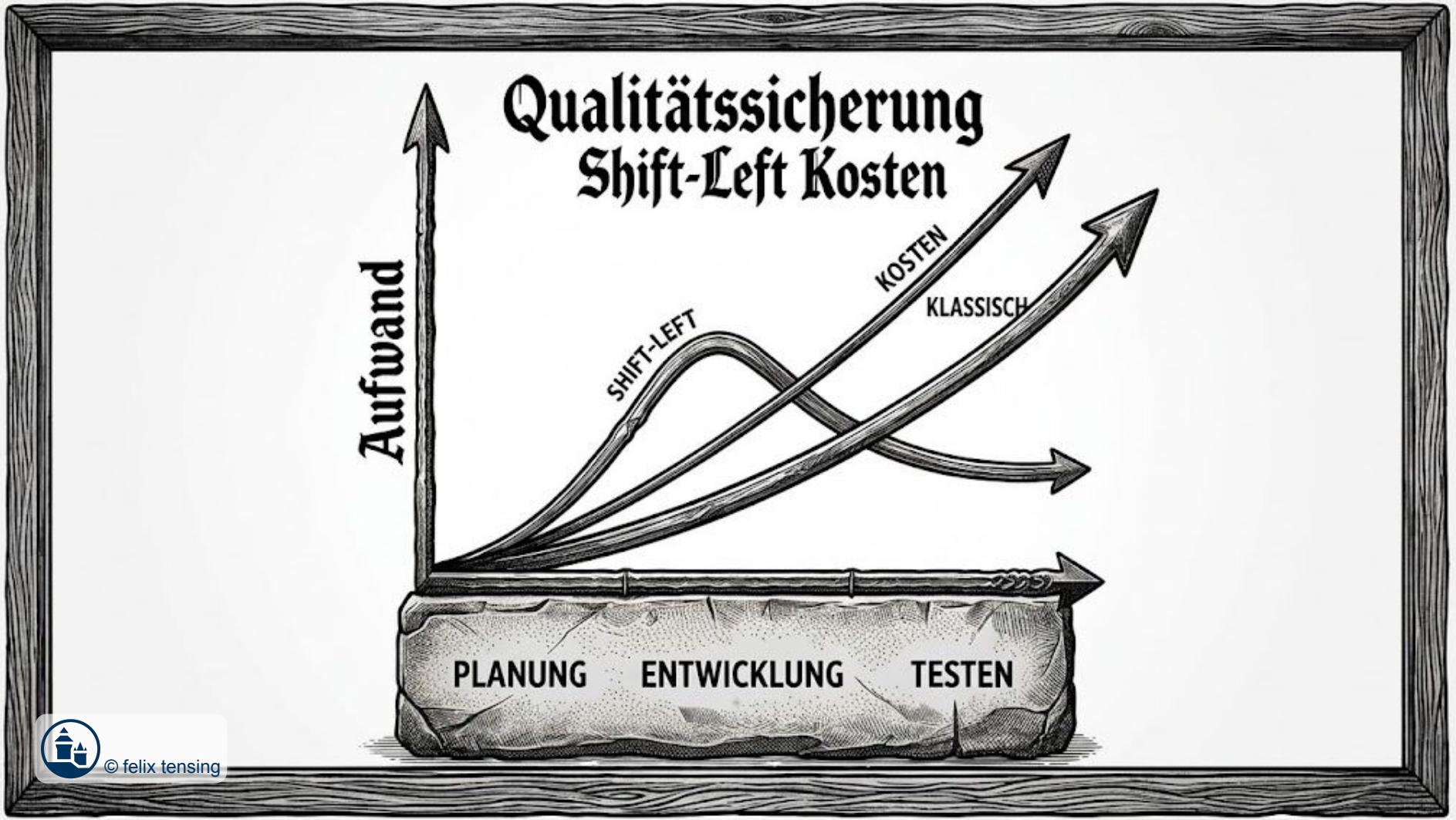
Softwareingenieur und Drachentöter im Zeichen der Burg

felix-spricht.de

Shift Left



© felix tensing



© felix tensing

Shift-Left - Mehr als Tools

Konzept	Fokus	Werkzeuge (u.a.)
Doing things right Stabilität / Effizienz	● Technische Korrektheit ● Effizienz ● Syntax ● Performance	● Unit Tests ● **Testautomatisierung** ● CI/CD ● SonarQube
Doing the right thing Nutzen / Relevanz	● Sinnhaftigkeit ● Usability ● Business Value ● Logikfehler	● Domain Driven Design ● Exploratives Testen ● Pair Programming ● Reviews



© felix tensing

Das BDD Drama



© felix tensing

Was ist BDD - Behaviour Driven Development

„BDD is a second-generation, agile, software development process that extends TDD by writing test cases in a natural language that non-technical stakeholders can understand.“

Dan North, Introducing BDD

Scenario: Account is in credit

Given the account is in credit
And the card is valid
And the dispenser contains cash
When the customer requests cash
Then ensure the account is debited
And ensure cash is dispensed
And ensure the card is returned



© felix tensing

Das Versprechen

- Testfälle werden in natürlicher Sprache formuliert
- BDD ist ein gesamter Prozess (nicht nur Test)

- Ziel: Living Documentation, die jeder lesen kann – vom PO über die Fachabteilung bis hin zum Entwickler.
- "Nicht-Techies" definieren das Fachverhalten, und der Test ist gleichzeitig die Anforderung. Keine Übersetzungsprobleme mehr, pure Transparenz, echtes Shift-Left.
- Dokumentation ist nicht nur Outcome, es ist auch Input.

"If you're using Cucumber solely for test automation, you're doing it wrong."
Aslak Hellesøy (Erfinder von Cucumber)



© felix tensing

Problem 1 - "non-technical stakeholders can understand"

1. Gherkin ist eine DSL, keine natürliche Sprache
   a. Arbeiten in extra Tools/Git/IDEs nötig
   b. Formulierungen benötigten Aufwand und sind Fehleranfällig (Actionword Reihenfolge)
2. In geschätzten 80–90 % aller Projekte schreiben am Ende doch nur Entwickler oder QA-Automatisierer die Gherkin-Dateien.

⇒ Es entsteht ein enormer Aufwand für ein Feature, das die Zielgruppe gar nicht nutzt.

„Cucumber is just an extra layer of bullshit that doubles the work for QAs.“
Reddit r/QualityAssurance:



© felix tensing

Problem 2: Cluecode of Hell

- Der Verbindungscode (Glue-Code/Step-Definitions) ist aufwendig zu Pflegen
- Bei Testfehlern sucht man erste Fehler im Gherkin, dann Fehler in der Implementierung
- Debug wird erschwert durch Indirektion
- Faktor Mensch: Arbeit die aufwendig/mühsam ist wird gescheut, minimalistisch erledigt

⇒ Das Ergebnis: Wir verwalten nur noch bürokratischen Overhead und verrotteten Testcode.



© felix tensing

Problem 3: Testszenario-Story-Dilemma

- BDD wird meistens auf User-Story-Ebene angewendet: Eine Story wird verfeinert, Akzeptanzkriterien formuliert und dafür 3-5 BDD-Szenarien geschrieben.
- Drei Sprints später kommt eine verwandte Story. Es werden wieder neue BDD-Szenarien geschrieben – viele davon durchlaufen dieselben Pfade (z. B. Registrierung, Login, Warenkorb befüllen).
- Das Ergebnis:
  - Test-Explosion: Niemand refactored nach dem Sprint die bestehende BDD-Suite, um Redundanzen zu eliminieren.
  - Verrottete Suites: Ändert sich ein grundlegender Ablauf, schlagen 40 Cucumber-Szenarien aus 10 verschiedenen vergangenen Sprints fehl.

BDD nur für Frontend ⇒ Test die eigentlich auf einer anderen Ebene sein sollten, werden im FE gemacht, da Teil der Story



© felix tensing

Problem 4: Testpyramiden-Inversion

- Weil es so einfach wirkt, Anforderungen in Freitext zu formulieren, schreiben Fachseite oder Entwickler Grenzwerttests direkt in die BDD-Szenarien

  `("Wenn Alter = 17, dann Fehler", "Wenn Alter = 18, dann Erlaubt", "Wenn E-Mail ohne @, dann Fehler").`
- Diese Logik gehört eigentlich in einen Unit-Test, der in 2 Millisekunden durchläuft.
- Stattdessen startet der BDD-Test jetzt 10-mal den echten Browser, tippt die Werte in Formularfelder ein und wartet auf die Server-Antwort.



© felix tensing

Die Utopie



© felix tensing

Der perfekte BDD Neuansatz

1. Echte natürliche Sprache für Testbeschreibung (Living Dokumentation)
2. k(l)eine technische Hürde (Jeder soll sich beteiligen können)
3. Testmanagement (keine Dopplungen)
4. Qualitätsprüfung (z.B. Testpyramide, Testslop verhindern)
5. Kein Vendor-Lockin



© felix tensing

AI mein Retter



© felix tensing

Der Game-Changer

- AI hat ein Verständnis natürlicher Sprache
  a. Synonyme
  b. Nuancen
  c. Kontext
- Regular Expressions, Keywords usw. werden obsolet
- Fachlichkeit steht im Fokus
- Testbeschreibung übersteht ein Refactoring.

Filtere die Tabelle nach grünen Autos

⇒ KI ist Compiler



© felix tensing

2. KI als Bindeglied

- Selbstlernend durch Anlegen von Instruktionen
- Funktioniert die Codegenerierung nicht, ist die Spezifiktion (Spec) ungenügend
- Technologieagnostisch (Playwright, RestAssured …)



© felix tensing

Skill: review-markdown-spec

⇒ eine Spec soll geprüft werden, bevor ein Playwright-Test entsteht oder geändert wird.

- Eingang: fertige Spec **oder** Testbeschreibung in natürlicher Sprache
- sicherstellen einer ID
- überführung in Spec-Format
- jeden Schritt live im Browser verifizieren
- Prüft die Spec als Source of Truth, NICHT den Code
- 8 Checks:
  - imperative Schritte
  - keine technischen Selektoren
  - 1 Aktion oder 1 Verifikation pro Zeile
  - konkrete Testdaten und Erwartungen

Unklare Specs erzeugen schlechte Tests



© felix tensing

Skill: generate-playwright-test-from-spec

⇒ aus einer geprüften Spec soll Playwright-Code entstehen.

- Implementiert nur das Spec-Verhalten; erfindet nichts dazu
- Jeder Assertion-Block trägt den ganzen Spec-Step-Text als Kommentar
- Selektor-Priorität:
  - Role/Test-ID ⇒ Label/Placeholder ⇒ sichtbarer Text ⇒ stabile IDs
- Neues App-Wissen wandert in `learnings.md`
- Keine statischen Waits und keine breiten Retries: auf beobachtbare Bedingungen warten
- Abgeschlossen erst, wenn der Test läuft oder ein Blocker dokumentiert ist.
- Regeneration statt Reparatur: Jede Spec-Änderung und jeder Testfehler führen zu einer kompletten Neugenerierung aus Spec

Der Test implementiert die Spec — nichts anderes.

© felix tensing

Skill: debug-playwright-test-failure

⇒ ein aus einer Spec generierter Test schlägt fehl, ist flaky oder liefert ein unerwartetes Ergebnis.

- Pflicht-Loop:
  - testausführung
  - Fehler/Trace/Screenshot/Spec-Step prüfen
  - Ursache klassifizieren
  - Aktion/Learning
  - Test neu erzeugen
- nach der Diagnose Test neu erzeugen, Basis ist Spec + `learnings.md`
- Test nie patchen
- Blocker dokumentieren; nach Behebung ebenfalls komplett neu generieren, nie die alte Fassung weiterverwenden

Diagnose am alten Test, Fix als Neugenerierung.

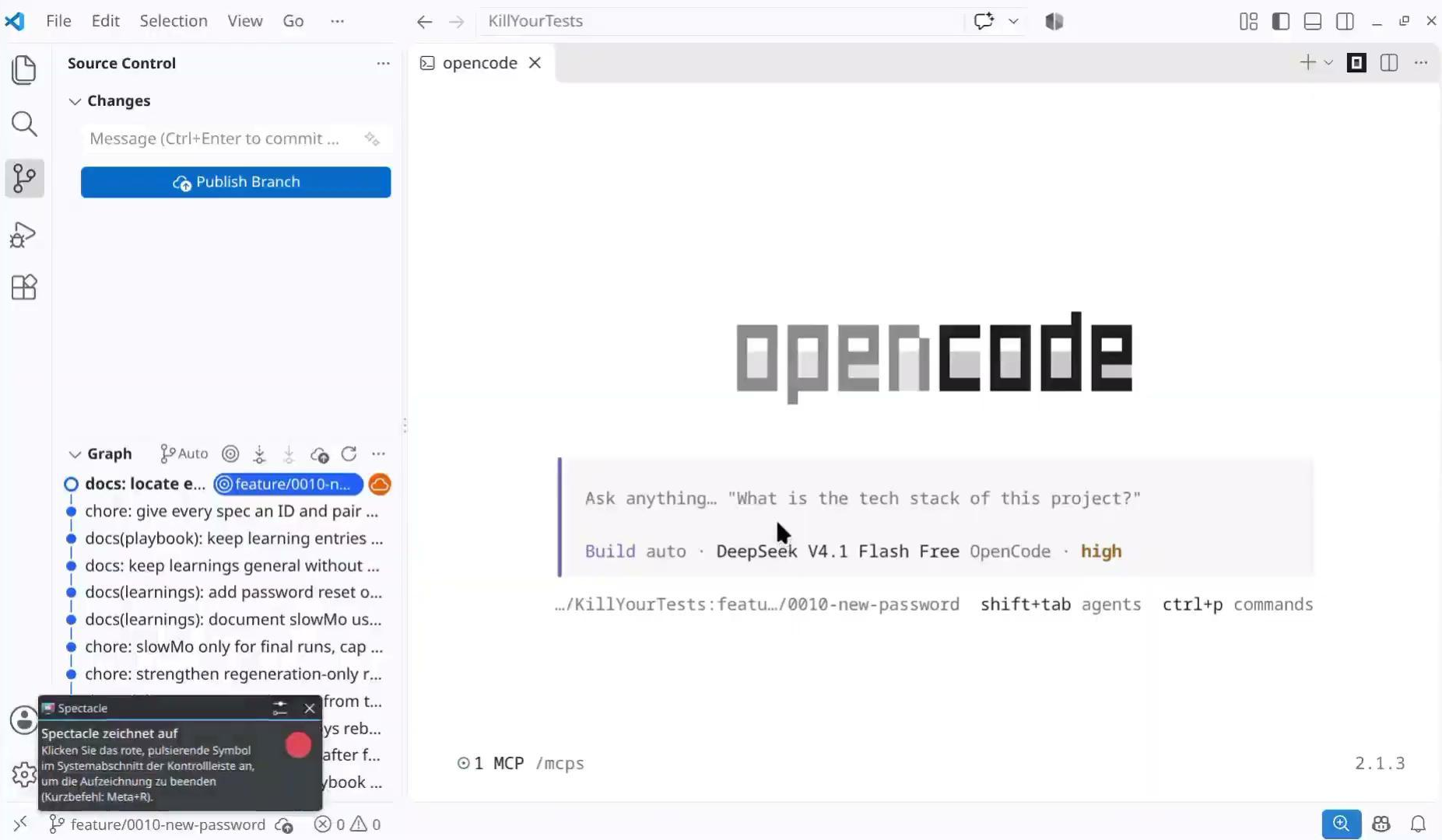


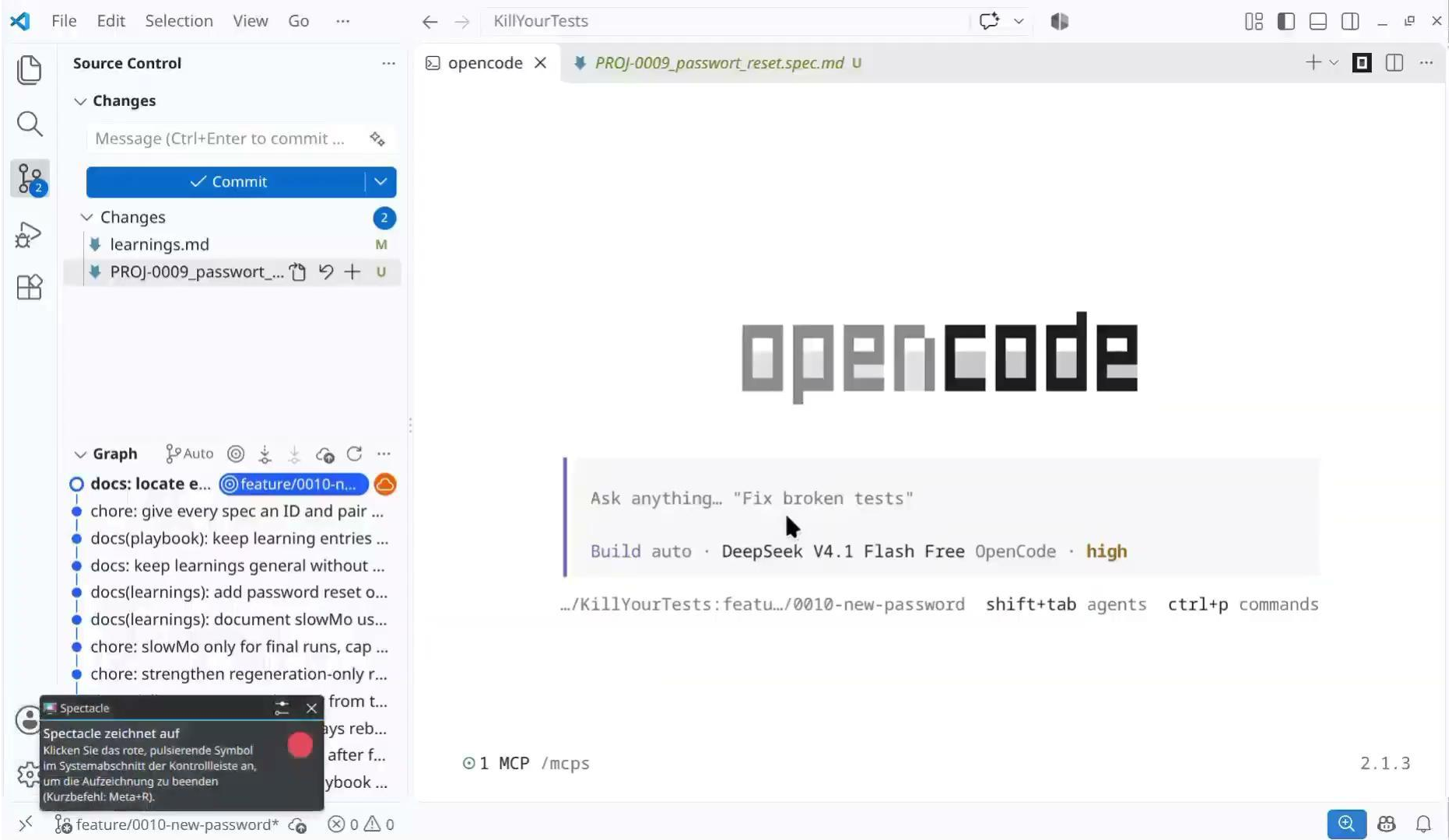
© felix tensing

Praxis



© felix tensing


File Edit Selection View Go
KillYourTests
Source Control
Changes
Message (Ctrl+Enter to commit ...
Publish Branch
opencode
Graph
Auto
docs: locate e...
feature/0010-n...
chore: give every spec an ID and pair ...
docs(playbook): keep learning entries ...
docs: keep learnings general without ...
docs(learnings): add password reset o...
docs(learnings): document slowMo us...
chore: slowMo only for final runs, cap ...
chore: strengthen regeneration-only r...
Spectacle
Spectacle zeichnet auf
Klicken Sie das rote, pulsierende Symbol im Systemabschnitt der Kontrollleiste an, um die Aufzeichnung zu beenden (Kurzbefehl: Meta+R).
opencode
Ask anything... "What is the tech stack of this project?"
Build auto · DeepSeek V4.1 Flash Free OpenCode · high
…/KillYourTests:featu…/0010-new-password shift+tab agents ctrl+p commands
1 MCP /mcps
2.1.3
feature/0010-new-password
0 0


File Edit Selection View Go
KillYourTests
Source Control
Changes
Message (Ctrl+Enter to commit ...
Commit
Changes 2
learnings.md M
PROJ-0009_passwort_... U
Graph Auto
docs: locate e... feature/0010-n...
chore: give every spec an ID and pair ...
docs(playbook): keep learning entries ...
docs: keep learnings general without ...
docs(learnings): add password reset o...
docs(learnings): document slowMo us...
chore: slowMo only for final runs, cap ...
chore: strengthen regeneration-only r...
Spectacle
Spectacle zeichnet auf
Klicken Sie das rote, pulsierende Symbol im Systemabschnitt der Kontrollleiste an, um die Aufzeichnung zu beenden (Kurzbefehl: Meta+R).
opencode
PROJ-0009_passwort_reset.spec.md U
opencode
Ask anything... "Fix broken tests"
Build auto · DeepSeek V4.1 Flash Free OpenCode · high
.../KillYourTests:featu.../0010-new-password shift+tab agents ctrl+p commands
1 MCP /mcps
2.1.3
feature/0010-new-password*
0 0

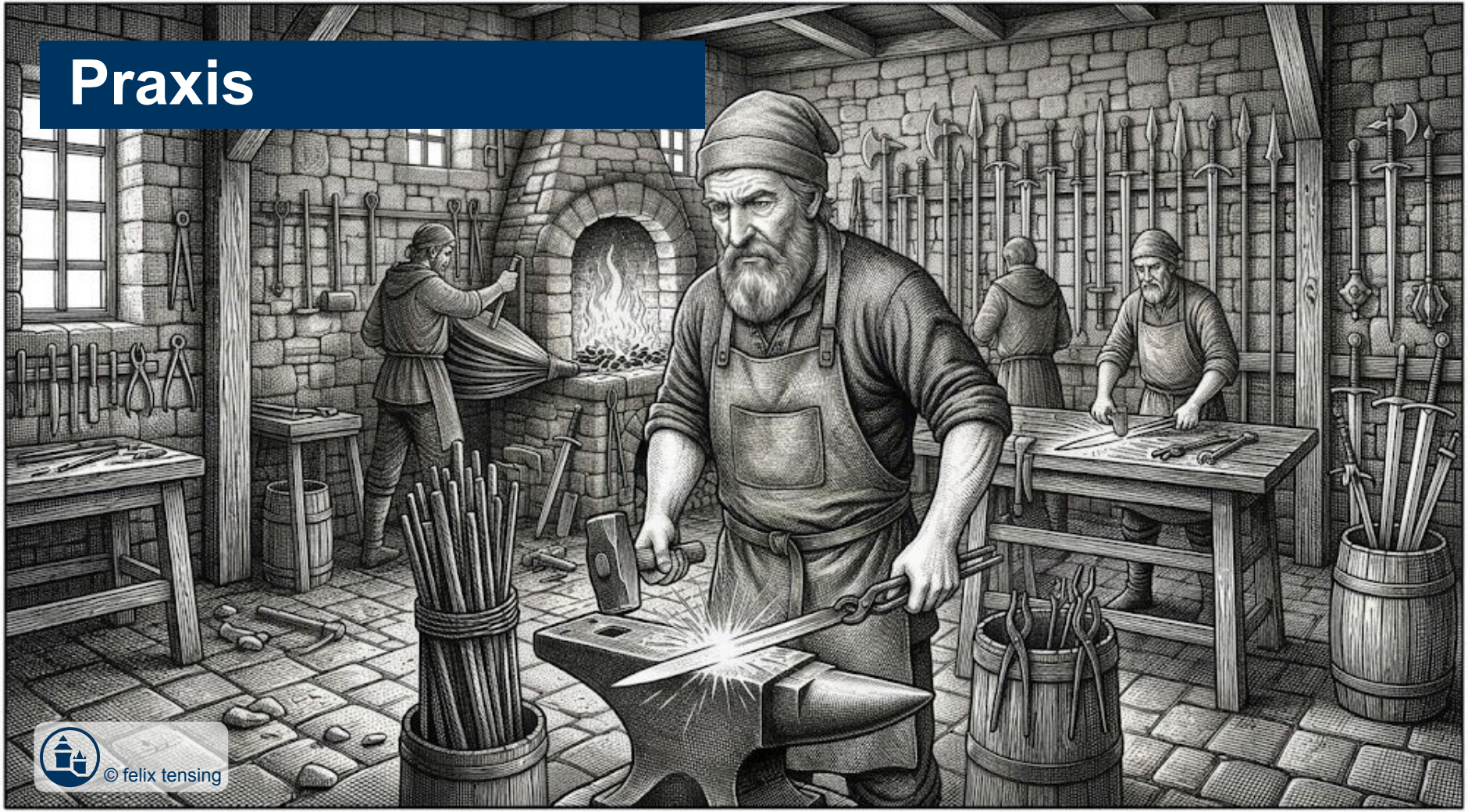
Warum "Töten" wir den Test?

1. Verhindern von "Drift"
    a. Die "Wahrheit" steht im Code
    b. Die Spec ist die fachliche Wahrheit
    c. ändern von Code ohne ändern der Spec verfälscht die Vereinfachung Spec = Code
2. Verifizieren der "Learnings"
3. Kein Einlesen/Debuggen in den Code



© felix tensing

Praxis



© felix tensing

Einsatz in der Praxis

1. **DoR:** Userstory beinhaltet Testbeschreibung
2. **Entwicklung:** Entwickler prüft mit KI bestehende Testbasis, ob ein Test erweitert werden kann/soll oder ein neuer entsteht
3. **DoD:** Geänderter / neue TestID wird in Story hinterlegt (Rückverfolgbarkeit), Code-Review auf Test
4. **Abnahme:** Spec ist Teil der Prüfung
5. **Dokumentation:** Spec, Testergebnis und Version werden ins Testreporting (aqua) übertragen (Szenario ist langlebiger als ein Test)

Nicht jede Story MUSS einen E2E Test haben (Testpyramide)



© felix tensing

Recap



© felix tensing

Vorteile

- Echte natürliche Sprache (BDD-Problem 1)
- Trennung fachliche Beschreibung und Testcode ohne "Glue Code" (BDD-Problem 2)
- KI unterstützt beim Testmanagement (BDD-Problem 3 und 4)
- Keine Abhängigkeiten zu proprietären (BDD) Frameworks
- Technologie Zweitrangig
- Kaum technisches Know-How nötig (Code-Review)
- KI erstellt Tests, aber Testcode ist statisch
- Wartungskosten viel geringer
- …



© felix tensing

und …

- Vision-Modelle (Statt Pixel-Vergleich)
- Testmanager Skill (keine versteckten Unittests, Abdeckung, Doppelungen)
- Automatisierte Behebung von kaputten Tests (wie dependency-bot)
- Fachliche Dokumentation mit Tests verbinden

⚠️ **Mißbrauchsgefahr! Richtigen Einsatz beachten!**



© felix tensing

Danke fürs Zuhören und erfolgreiches Drachentöten!

linkedin.com/in/felix-tensing-nbg



felix-spricht.de



JUG Feedback



Ich freue mich auf Gespräche und Feedback!

© felix tensing

Test Promt PROJ-0009

Vorbedingung: Die Sprache der Anwendung ist auf Deutsch eingestellt.
Auflösung: Mobile
Szenario:

- Ein neuer Nutzer registriert sich mit einer eindeutigen E-Mail-Adresse, dem Passwort SuperSafe123!, der Sicherheitsfrage „Zweitname deines ältesten Geschwisters?" und der Antwort „Alex".
- Nach der Registrierung öffnet er die Anmeldeseite, klickt auf „Passwort vergessen?" und gibt seine E-Mail-Adresse ein.
- Er beantwortet die Sicherheitsfrage mit „Alex" und legt das neue Passwort NewPass456! fest.
- Daraufhin wird ihm die Meldung „Dein Passwort wurde erfolgreich geändert." angezeigt.
- Jetzt meldet er sich mit dem neuen Passwort an und sieht das Warenkorb-Symbol oben rechts in der Navigationsleiste.
- Zum Schluss meldet er sich wieder ab und versucht, sich mit dem alten Passwort SuperSafe123! anzumelden. Das gelingt nicht und es erscheint die Fehlermeldung „Email-Adresse oder Passwort ungültig."

© felix tensing